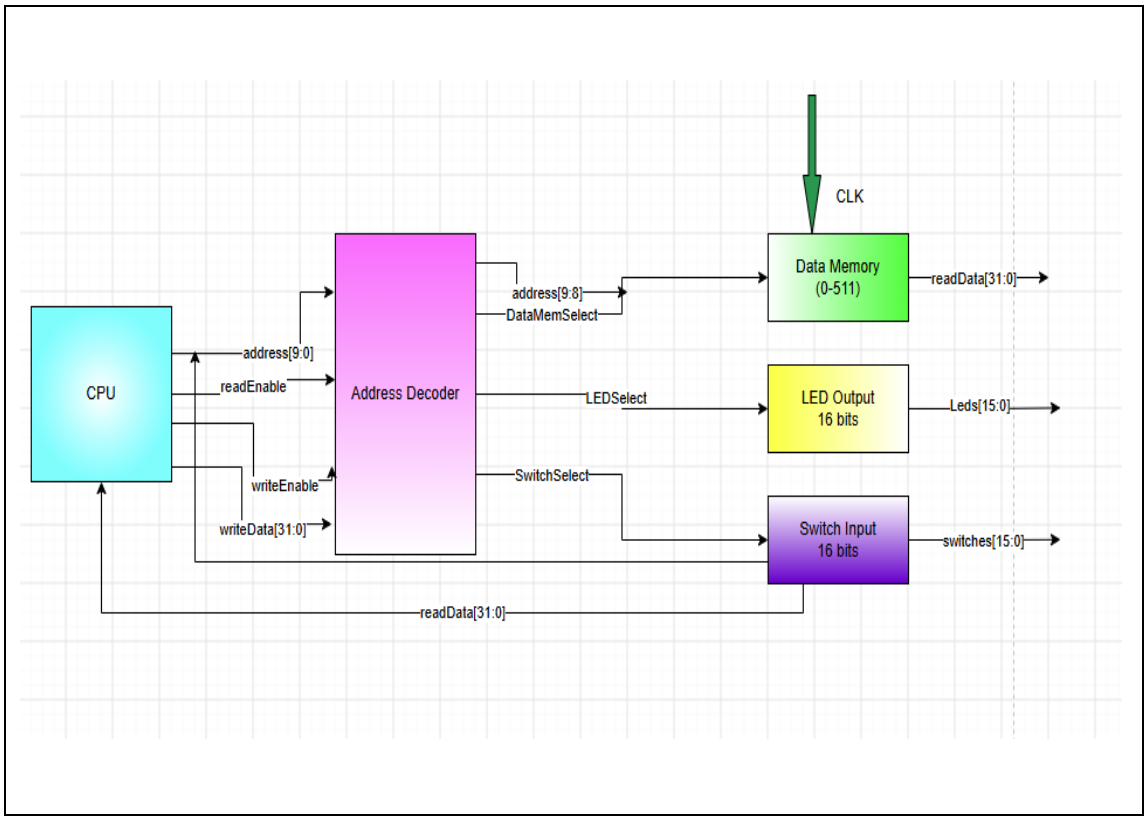
# Lab 08 – Worksheet

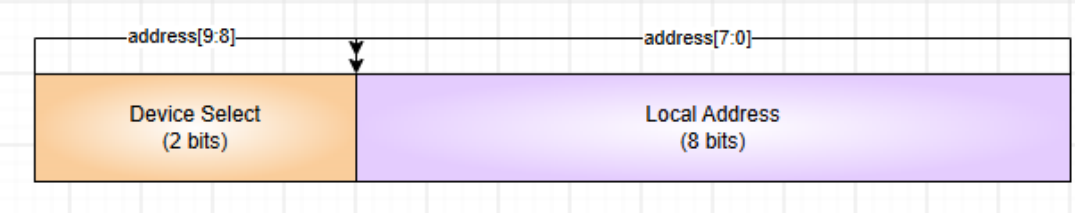
| | | |
|---|---|---|
| Name:Arsalan Rashid<br>Abdullah Rasool<br>Wasay Ali | ID:ar12008<br>ar11014<br>wa11538 | Section:T2 |

**Note:** Assumptions and logics should be explained separately in tasks after the task results.

## Task 1

*Block Diagram*
*a) Data Memory Module*
*b) Address Decoder*
*c) Top Module*






| Address[9:8] | Address Range in decimal | Device Used | Enable Signal |
|---|---|---|---|
| 00 | 0-255 | Data Memory | DataMemSelect |
| 01 | 255-511 | Data Memory | DataMemSelect |
| 10 | 512-767 | LEDs | LEDSelect |
| 11 | 768-1023 | Switch | SwitchSelect |

*Comments*

*Although the processor uses a 32 bit address, this lab only requires a local address space of 1024 locations. Therefore, only the lower 10 bits, address[9:0] are used for address decoding while address[31:10] are unused. This simplifies the address decoding hardware because the system only needs to distinguish between DataMemory, LEDs and Switches within the required address range. The 512- location Data Memory uses addresses 0-511 while the remaining address rages are used for the LED and Switch interfaces.*

## Task 2

*Modules for **DataMemory, AddressDecoder, AddressDecoderTop, Switches and Leds.***

### *DataMemory.v*

```
`timescale 1ns / 1ps

module DataMemory(
    input clk,
    input MemWrite,
    input MemRead,

    input [8:0] address,
    input [31:0] write_data,

    output [31:0] read_data
);

    reg [31:0] memory [0:511];
```

```
    always @(posedge clk) begin
        if (MemWrite)
            memory[address] <= write_data;
    end

    assign read_data = MemRead ? memory[address] : 32'b0;

endmodule
```

## *AddressDecoder.v*

```
`timescale 1ns / 1ps

module AddressDecoder(
    input [31:0] address,

    input readEnable,
    input writeEnable,

    output reg DataMemRead,
    output reg DataMemWrite,

    output reg LEDWrite,
    output reg SwitchReadEnable
);

always @(*) begin

    DataMemRead     = 1'b0;
    DataMemWrite    = 1'b0;
    LEDWrite        = 1'b0;
    SwitchReadEnable = 1'b0;


    if (address[9] == 1'b0) begin

        DataMemRead  = readEnable;
        DataMemWrite = writeEnable;

    end

    else if (address[9:8] == 2'b10) begin

        LEDWrite = writeEnable;
```

```
    end

    else if (address[9:8] == 2'b11) begin

        SwitchReadEnable = readEnable;

    end

end

endmodule
```

## Switches.v

```
`timescale 1ns /1ps

module switches(
    input clk,
    input rst,

    input[31:0] writeData,
    input writeEnable,
    input readEnable,
    input [29:0] memAddress,

    output reg [31:0] readData,
    output reg [15:0] leds
);

always @(posedge clk or posedge rst) begin
    if (rst) begin
        leds <= 16'd0;
        readData <= 32'd0;
    end

    else begin
        if (writeEnable)
            leds <= writeData[15:0];
        readData <= 32'd0;
    end

 end

endmodule
```

## Leds.v

```
`timescale 1ns / 1ps

module leds(
    input clk,
    input rst,

    input [15:0] btns,
    input [31:0] writeData,
    input writeEnable,
    input readEnable,
    input [29:0] memAddress,
    input [15:0] switches,

    output reg [31:0] readData
    );

always @(posedge clk or posedge rst) begin
    if (rst) begin
        readData <= 31'd0;
    end

    else begin
        if (readEnable)
            readData <= {16'd0, switches};
        else
            readData <= 32'd0;
    end

 end

endmodule
```

## *AddressDecoderTop.v*

```
`timescale 1ns / 1ps

module addressDecoderTop(

    input clk,
    input rst,

    input [31:0] address,

    input readEnable,
    input writeEnable,
```

```
    input [31:0] writeData,

    input [15:0] switches,

    output [31:0] readData,
    output [15:0] leds

);


wire DataMemRead;
wire DataMemWrite;

wire LEDWrite;
wire SwitchReadEnable;

wire [31:0] memoryReadData;
wire [31:0] switchReadData;


AddressDecoder decoder (

    .address(address),

    .readEnable(readEnable),
    .writeEnable(writeEnable),

    .DataMemRead(DataMemRead),
    .DataMemWrite(DataMemWrite),

    .LEDWrite(LEDWrite),
    .SwitchReadEnable(SwitchReadEnable)

);

DataMemory data_memory (

    .clk(clk),

    .MemWrite(DataMemWrite),
    .MemRead(DataMemRead),

    .address(address[8:0]),

    .write_data(writeData),

    .read_data(memoryReadData)
```

```
);

switches led_interface (

    .clk(clk),
    .rst(rst),

    .writeData(writeData),

    .writeEnable(LEDWrite),
    .readEnable(1'b0),

    .memAddress(address[29:0]),

    .readData(),

    .leds(leds)

);

leds switch_interface (

    .clk(clk),
    .rst(rst),

    .btns(16'd0),

    .writeData(32'd0),
    .writeEnable(1'b0),

    .readEnable(SwitchReadEnable),

    .memAddress(address[29:0]),

    .switches(switches),

    .readData(switchReadData)

);

assign readData =
       DataMemRead
       ? memoryReadData

       : SwitchReadEnable
       ? switchReadData

       : 32'b0;
```

```
endmodule
```

Testbench:

```
`timescale 1ns / 1ps

module MemorySystem_tb;


reg clk;
reg rst;

reg [31:0] address;

reg readEnable;
reg writeEnable;

reg [31:0] writeData;

reg [15:0] switches;


wire [31:0] readData;
wire [15:0] leds;


addressDecoderTop uut (

    .clk(clk),
    .rst(rst),

    .address(address),

    .readEnable(readEnable),
    .writeEnable(writeEnable),

    .writeData(writeData),

    .switches(switches),

    .readData(readData),
    .leds(leds)

);
```

```
initial begin

    clk = 0;

    forever #5 clk = ~clk;

end


initial begin

    // Initial values
    rst = 1;

    address = 0;

    readEnable = 0;
    writeEnable = 0;

    writeData = 0;

    switches = 16'h0000;


    #20;

    rst = 0;




    address = 32'd10;

    writeData = 32'hDEADBEEF;

    writeEnable = 1;
    readEnable = 0;

    #10;

    writeEnable = 0;


    readEnable = 1;

    #10;
```

```
$display(
   "MEMORY READ: Address=%d Data=%h",
   address,
   readData
);


readEnable = 0;


address = 32'h00000210;

writeData = 32'h000000A5;

writeEnable = 1;

#10;

writeEnable = 0;

#10;

$display(
   "LED WRITE: LEDs=%h",
   leds
);

switches = 16'h5A3C;

address = 32'h00000350;

readEnable = 1;

#10;

$display(
   "SWITCH READ: Switches=%h ReadData=%h",
   switches,
   readData
);


readEnable = 0;


#20;

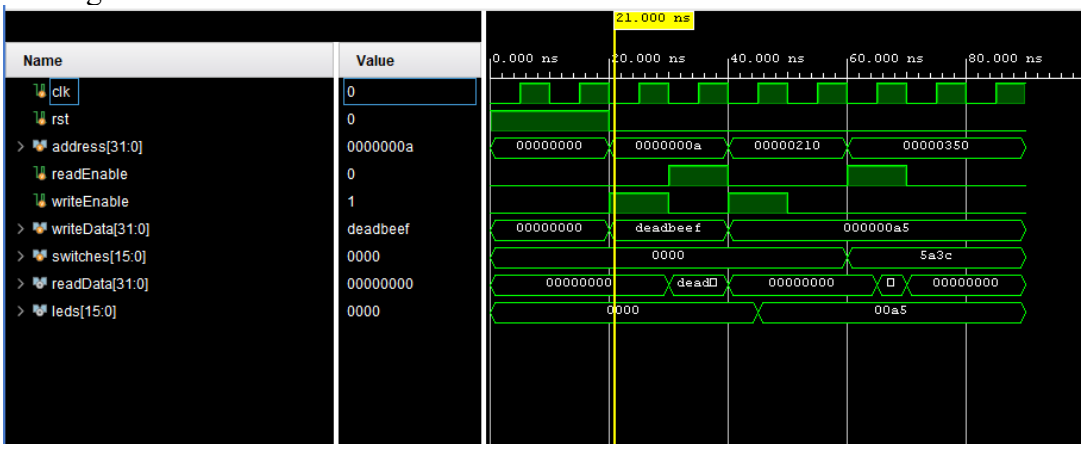
$finish;
```

```
end

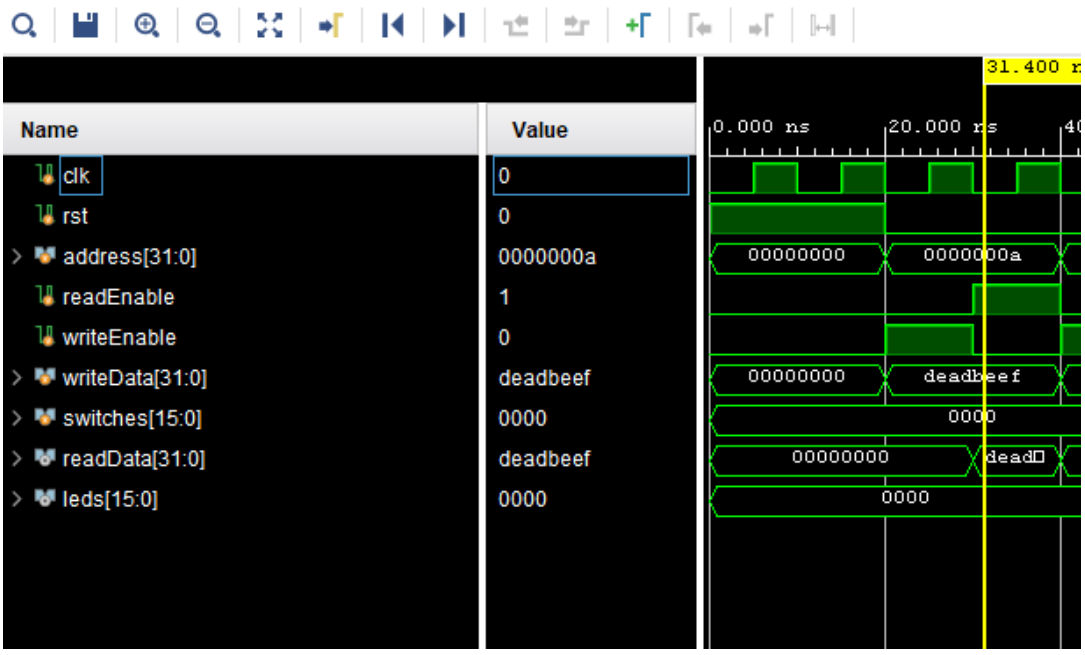
endmodule
```

Waveforms:

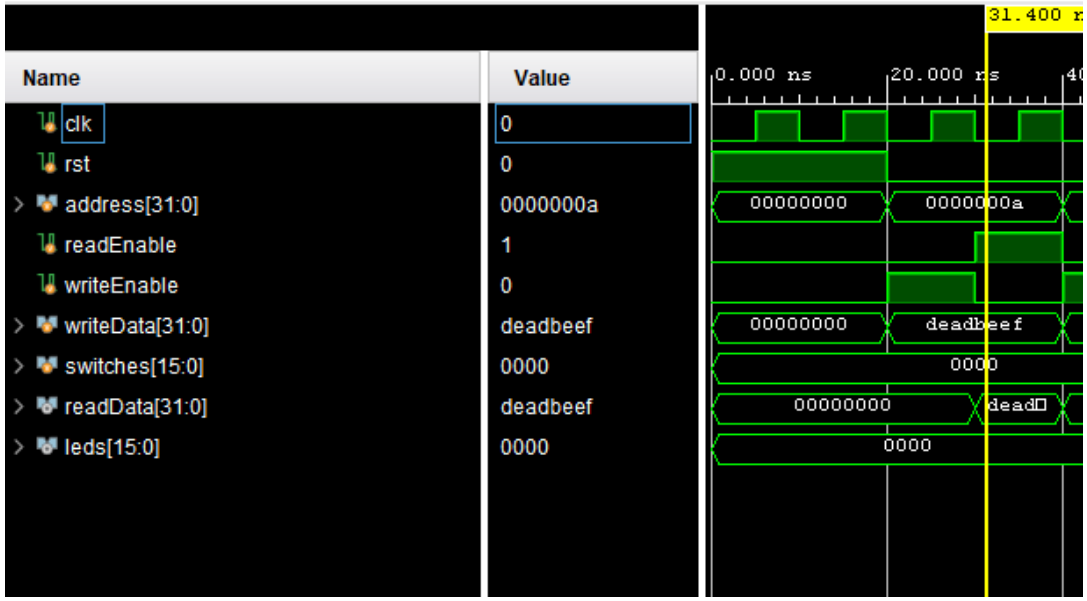
Writing deadbeef to address 0xa



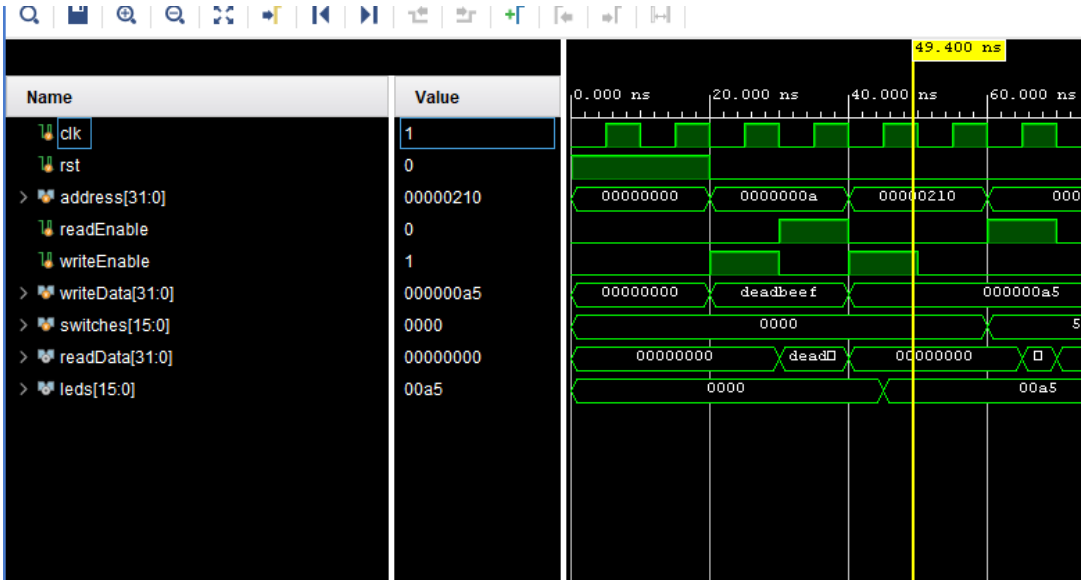

Reading address from address 0xa – it is deadbeef (Data written successfully)



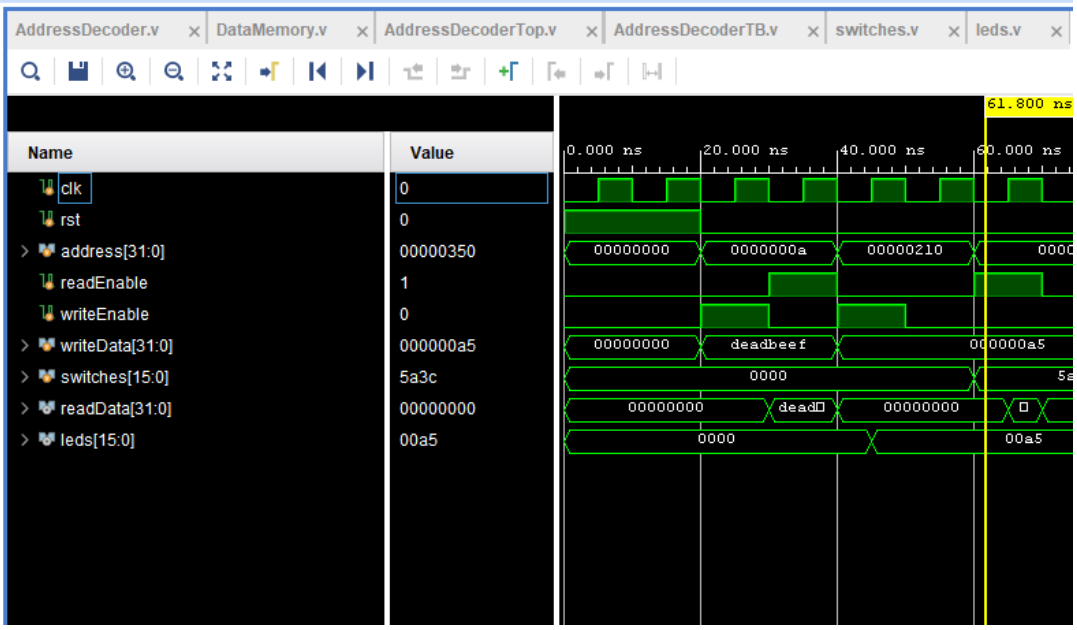

Writing a5 into address 0x210 to show If LED output works:



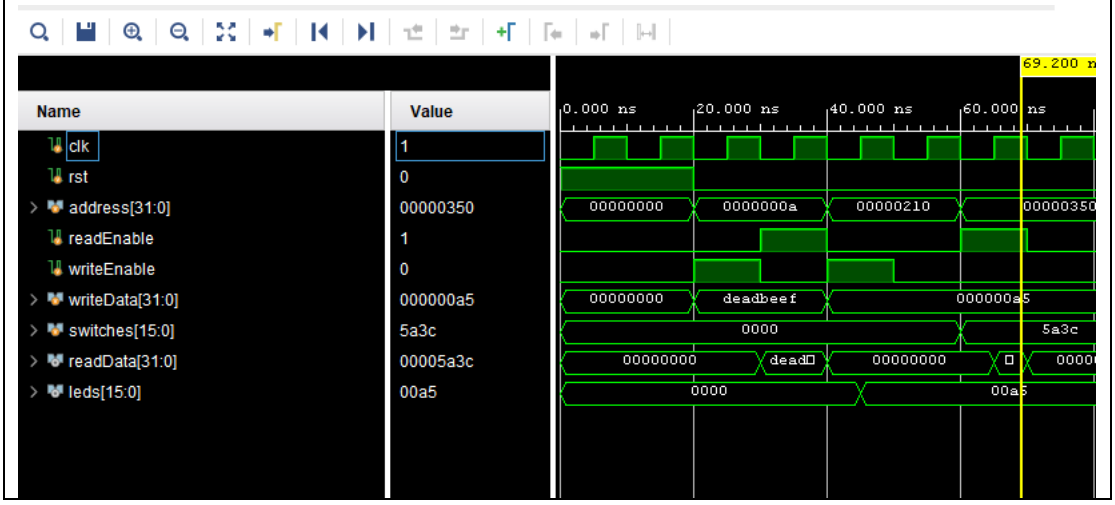

LED shows a5 hence LED WORKS




Going into address 0x350 and writing input through switch. Input by switch is 5a3c



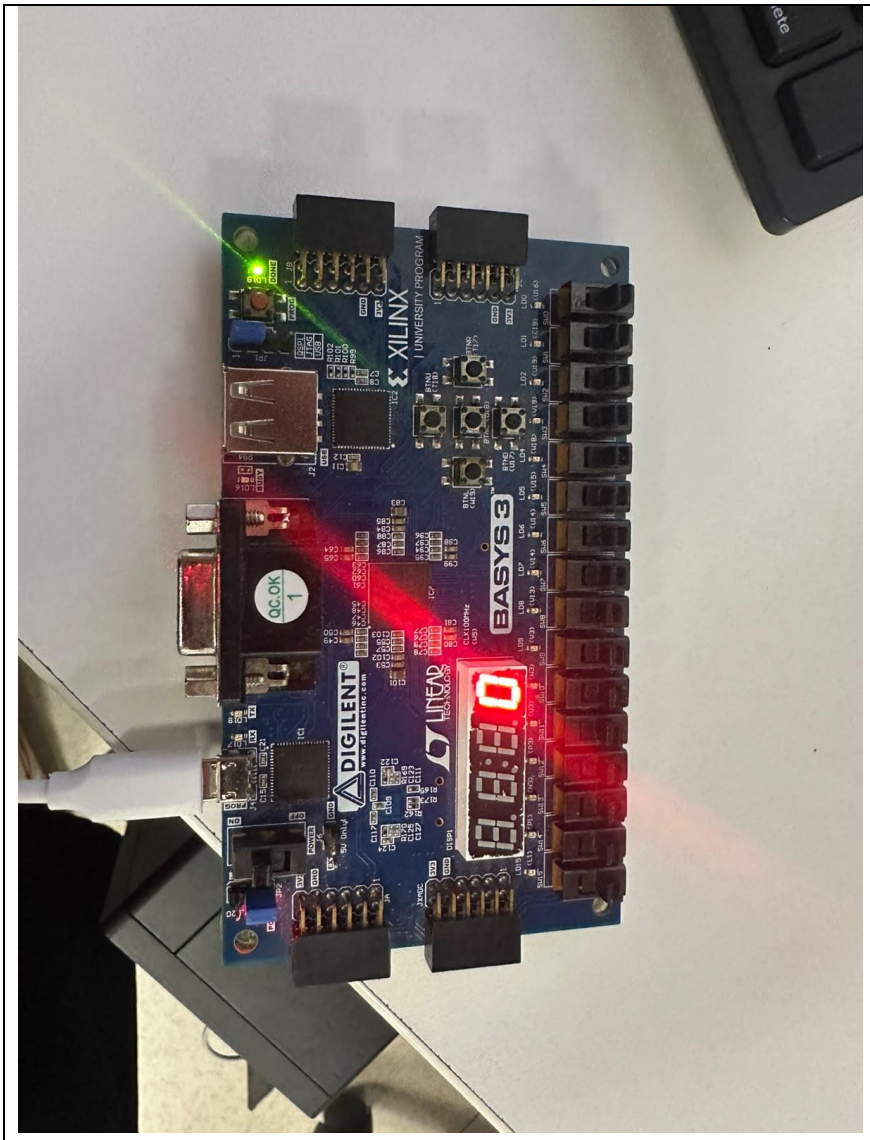

Now confirming through ReadData if it was written successfully – readData shows 5a3c hence PERFECT!



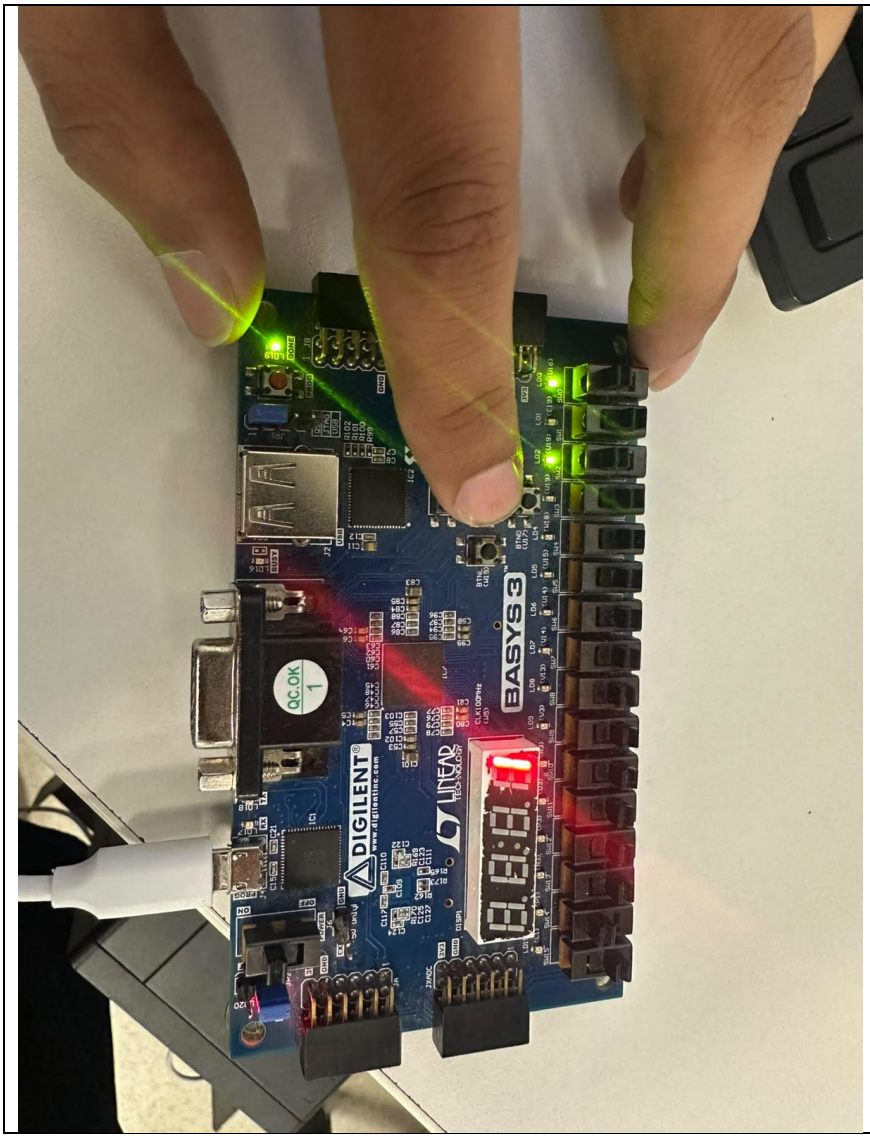

## Task 3

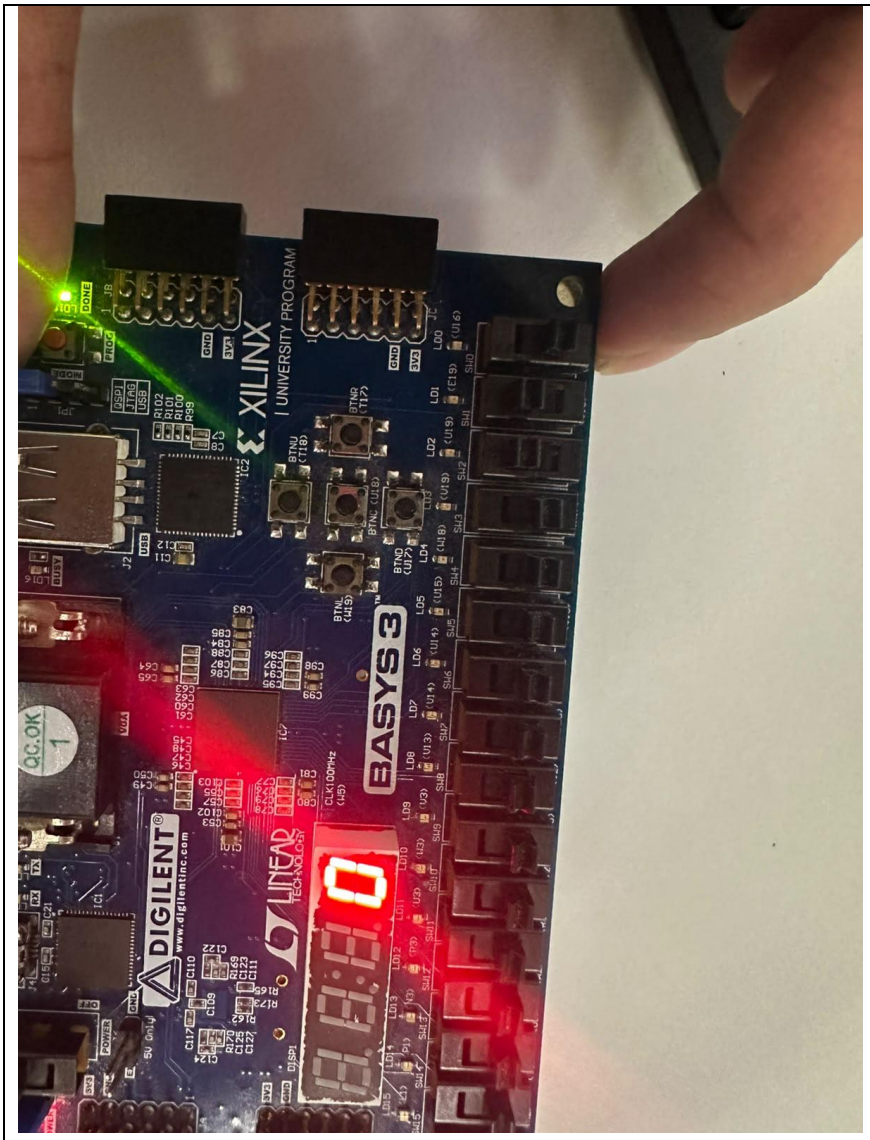
*FSM*

| 0 State: No Write – No Memory – IDLE STATE |
|---|
| |

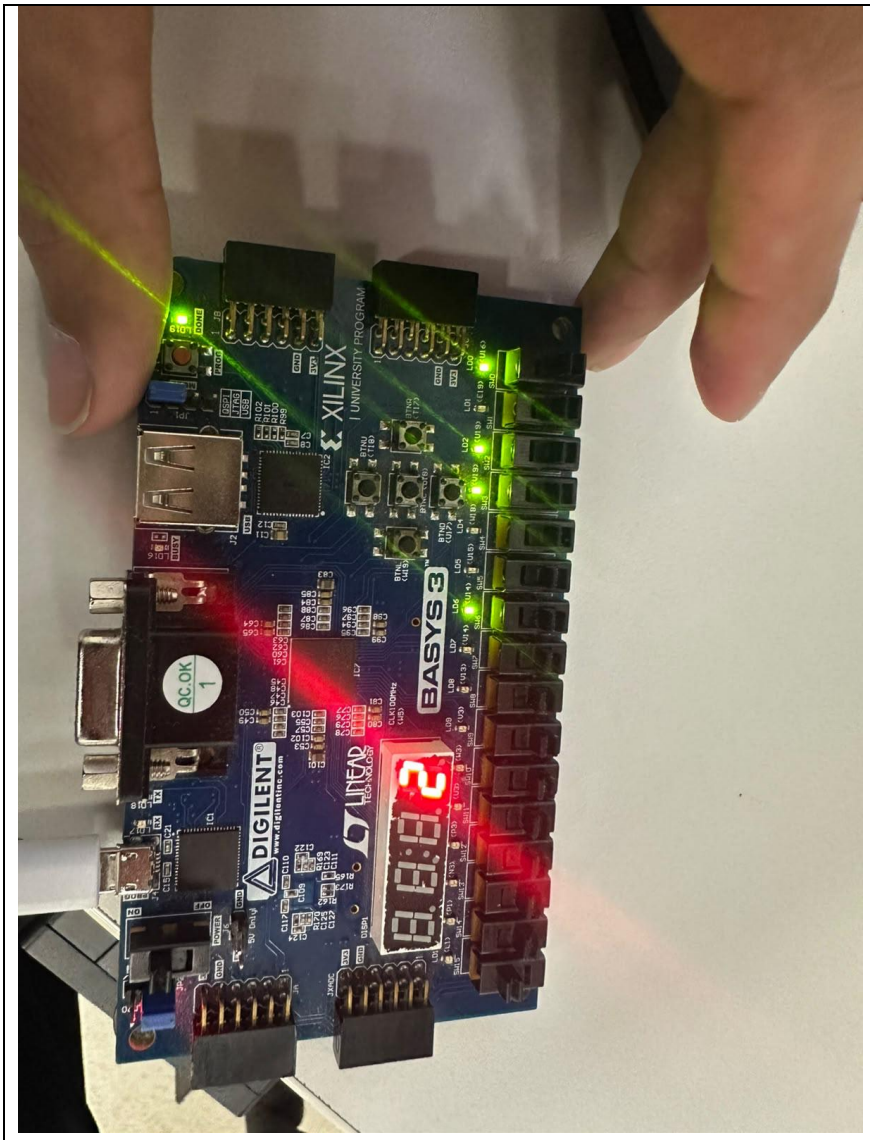



WRITING TO MEMORY - 0110 and Pressing bTNC to save it

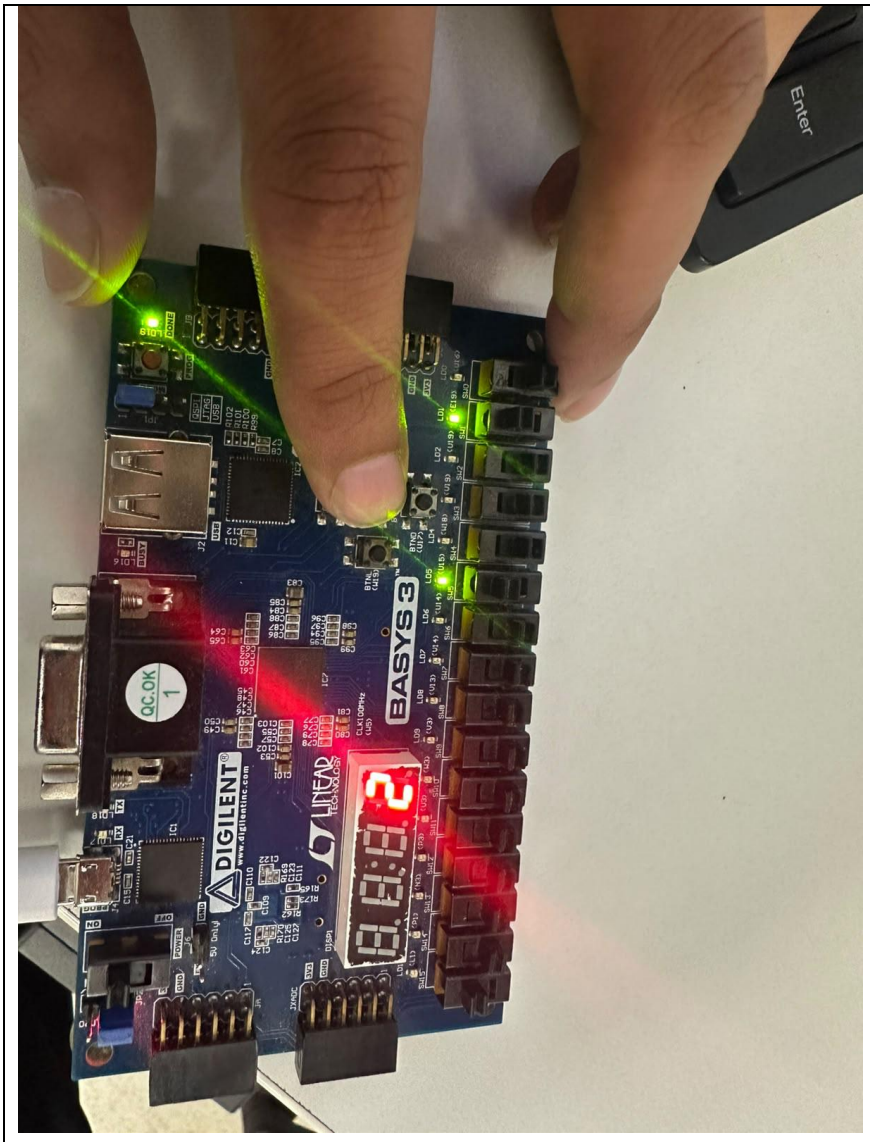

BASYS 3
DIGILENT
QC.OK



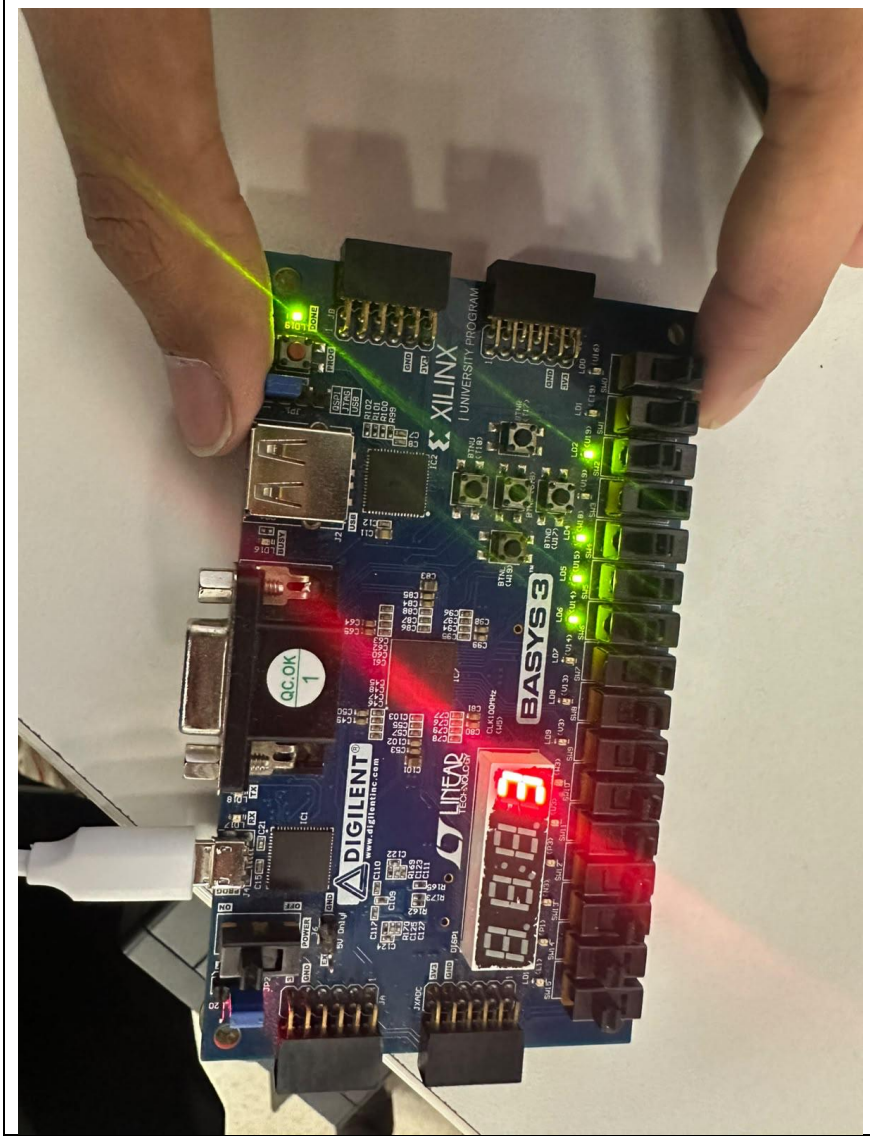

STATE 2 – LOAD MEMORY – checking saved state at 0110 turns on LEDS with the data that was written which was also 0110




Wrote again and went to another memory address to verify multiple save locations:




STATE 3: LEDS REFLECTS WRITE




CONSTRAINT FILE

CLOCK - 100 MHz
set_property PACKAGE_PIN W5 [get_ports clk] set_property IOSTANDARD LVCMOS33 [get_ports clk] create_clock -period 10.000 -name sys_clk [get_ports clk]

## CENTER BUTTON - RESET

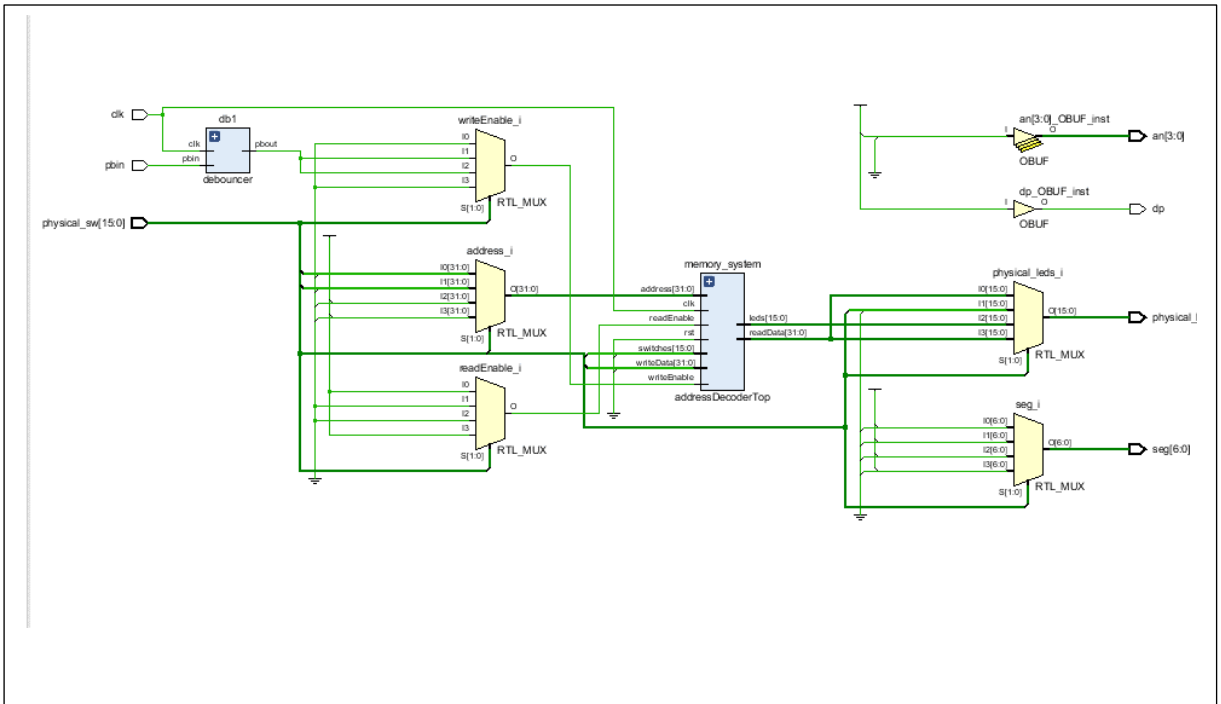
set_property PACKAGE_PIN U18 [get_ports pbin] set_property IOSTANDARD LVCMOS33 [get_ports pbin]
set_property PACKAGE_PIN V17 [get_ports {physical_sw[0]}] set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[0]}]
set_property PACKAGE_PIN V16 [get_ports {physical_sw[1]}] set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[1]}]

```
set_property PACKAGE_PIN W16 [get_ports {physical_sw[2]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_sw[2]}]
set_property PACKAGE_PIN W17 [get_ports {physical_sw[3]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_sw[3]}]
set_property PACKAGE_PIN W15 [get_ports {physical_sw[4]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_sw[4]}]
set_property PACKAGE_PIN V15 [get_ports {physical_sw[5]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_sw[5]}]
set_property PACKAGE_PIN W14 [get_ports {physical_sw[6]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_sw[6]}]
set_property PACKAGE_PIN W13 [get_ports {physical_sw[7]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_sw[7]}]
set_property PACKAGE_PIN V2 [get_ports {physical_sw[8]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_sw[8]}]
set_property PACKAGE_PIN T3 [get_ports {physical_sw[9]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_sw[9]}]
set_property PACKAGE_PIN T2 [get_ports {physical_sw[10]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_sw[10]}]
set_property PACKAGE_PIN R3 [get_ports {physical_sw[11]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_sw[11]}]
set_property PACKAGE_PIN W2 [get_ports {physical_sw[12]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_sw[12]}]
set_property PACKAGE_PIN U1 [get_ports {physical_sw[13]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_sw[13]}
set_property PACKAGE_PIN T1 [get_ports {physical_sw[14]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_sw[14]}]
set_property PACKAGE_PIN R2 [get_ports {physical_sw[15]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_sw[15]}]
set_property PACKAGE_PIN U16 [get_ports {physical_leds[0]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_leds[0]}]
set_property PACKAGE_PIN E19 [get_ports {physical_leds[1]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_leds[1]}]
set_property PACKAGE_PIN U19 [get_ports {physical_leds[2]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_leds[2]}]
set_property PACKAGE_PIN V19 [get_ports {physical_leds[3]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_leds[3]}]
set_property PACKAGE_PIN W18 [get_ports {physical_leds[4]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_leds[4]}]
set_property PACKAGE_PIN U15 [get_ports {physical_leds[5]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_leds[5]}]
set_property PACKAGE_PIN U14 [get_ports {physical_leds[6]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_leds[6]}]
set_property PACKAGE_PIN V14 [get_ports {physical_leds[7]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_leds[7]}]
set_property PACKAGE_PIN V13 [get_ports {physical_leds[8]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_leds[8]}]
set_property PACKAGE_PIN V3 [get_ports {physical_leds[9]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_leds[9]}]
```

```
set_property PACKAGE_PIN W3 [get_ports {physical_leds[10]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_leds[10]}]
set_property PACKAGE_PIN U3 [get_ports {physical_leds[11]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_leds[11]}]
set_property PACKAGE_PIN P3 [get_ports {physical_leds[12]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_leds[12]}]
set_property PACKAGE_PIN N3 [get_ports {physical_leds[13]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_leds[13]}]
set_property PACKAGE_PIN P1 [get_ports {physical_leds[14]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_leds[14]}]
set_property PACKAGE_PIN L1 [get_ports {physical_leds[15]}] set_property
IOSTANDARD LVCMOS33 [get_ports {physical_leds[15]}]
set_property PACKAGE_PIN W7 [get_ports {seg[0]}] set_property IOSTANDARD
LVCMOS33 [get_ports {seg[0]}] set_property PACKAGE_PIN W6 [get_ports
{seg[1]}] set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
set_property PACKAGE_PIN U8 [get_ports {seg[2]}] set_property IOSTANDARD
LVCMOS33 [get_ports {seg[2]}] set_property PACKAGE_PIN V8 [get_ports
{seg[3]}] set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {seg[4]}] set_property IOSTANDARD
LVCMOS33 [get_ports {seg[4]}] set_property PACKAGE_PIN V5 [get_ports
{seg[5]}] set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
set_property PACKAGE_PIN U7 [get_ports {seg[6]}] set_property IOSTANDARD
LVCMOS33 [get_ports {seg[6]}]
set_property PACKAGE_PIN V7 [get_ports dp] set_property IOSTANDARD
LVCMOS33 [get_ports dp]
set_property PACKAGE_PIN U2 [get_ports {an[0]}] set_property IOSTANDARD
LVCMOS33 [get_ports {an[0]}] set_property PACKAGE_PIN U4 [get_ports
{an[1]}] set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}] set_property
PACKAGE_PIN V4 [get_ports {an[2]}] set_property IOSTANDARD LVCMOS33
[get_ports {an[2]}] set_property PACKAGE_PIN W4 [get_ports {an[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
```

RTL SCHEMATICS:

OBJ



# Lab 08 – Design and FPGA Implementation of Memory System with Address Decoding for a Single-Cycle RISC-V Processor

## Points Distribution

| Task No. | LR2 Code | LR 5 Results | LR11 Design | AR7 |
|---|---|---|---|---|
| Task 1 | - | - | /20 | /20 |
| Task 2 | /20 | /15 | | |
| Task 3 | /10 | /15 | | |
| Total Points | /80 | | | /20 |
| CLO Mapped | CLO 1 | | | |

*For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.*

| # | Assessment Elements | Level 1: Unsatisfactory Points 0-1 | Level 2: Developing Points 2 | Level 3:Good Points 3 | Level 4:Exemplary Points 4 |
|---|---|---|---|---|---|
| LR2 | Program/Code / Simulation Model/ Network Model | Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software. | Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software. | Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine. | Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software. |
| LR5 | Results & Plots | Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner. | Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear. | All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing. | Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic. |
| AR9 | Report | All the in-lab tasks are not included in report and / or the report is submitted too late. | Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date. | Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely. | Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables. |